

Term Project – Proposal

Gym Management System

Dozent: Stephan Bökelmann · sboekermann@ep1.rub.de · Technische Hochschule Georg Agricola, Bochum

Author: Eda Bodo Ghislaine Charline

Module: Introduction to Database Management Systems · THGA Bochum

Submitted to: Stephan Bökelmann · sboekermann@ep1.rub.de

Target size: about 6 pages

This proposal presents the Gym Management System project. The following sections describe the application domain, database design, REST API, frontend prototype, deployment strategy and testing plan.

1 Application domain and motivation

The proposed application is a **Gym Management System** for small and medium-sized fitness centres. The system provides a central platform to manage members, trainers, fitness courses, memberships and payments.

Many gyms still organise their daily work using spreadsheets or paper documents. This often leads to duplicate data, forgotten membership payments and difficulties when checking course availability. The proposed system stores all information in a PostgreSQL database and provides an easy-to-use web interface connected to a FastAPI backend.

The main users of the system are gym employees and administrators. Employees can register new members, manage memberships, create fitness courses and assign trainers. Administrators can monitor payments and generate reports about active members and course participation.

Usage scenario 1: A new customer visits the gym. An employee creates a member account, selects a membership plan and registers the customer for two fitness courses.

Usage scenario 2: At the beginning of each month, the administrator reviews all active memberships and checks which members have not yet paid their monthly fee.

2 Data model – ER sketch

Figure 1 shows the Entity-Relationship (ER) model of the Gym Management System. The main entities are Member, Trainer, Course, Membership, Payment and CourseRegistration. Each table has a primary key for unique identification, while foreign keys define the relationships between the entities. Members can have memberships, make payments and register for courses. Trainers are assigned to courses, and the CourseRegistration table represents the many-to-many relationship between members and courses.

3 From model to schema

- **Member**(member_id, first_name, last_name, email, phone, date_of_birth)
- **Trainer**(trainer_id, first_name, last_name, email, phone, specialization)
- **Course**(course_id, *trainer_id*, course_name, schedule, capacity)
- **Membership**(membership_id, *member_id*, membership_type, start_date, end_date)
- **Payment**(payment_id, *member_id*, amount, payment_date, payment_method)
- **CourseRegistration**(registration_id, *member_id*, *course_id*, registration_date)

Normalisation. The database schema is in Third Normal Form (3NF). Each table stores information about a single entity. primary keys uniquely identify each record, while foreign keys represent relationships between tables. This design minimizes redundancy and improves data consistency.

4 API design

Method	Path	Purpose
GET	/members	Returns all members.
GET	/courses	Returns all courses with trainer information (JOIN).
GET	/payments	Returns all payments.
POST	/members	Creates a new member (requires X-API-Key).
POST	/courses	Creates a new course (requires X-API-Key).
POST	/payments	Adds a payment (requires X-API-Key).
POST	/registrations	Registers a member for a course (requires X-API-Key).
PUT	/members/{id}	Updates a member (requires X-API-Key).
DELETE	/members/{id}	Deletes a member (requires X-API-Key).

The endpoint `/courses` performs a JOIN between the Course and Trainer tables. All POST, PUT and DELETE endpoints require an X-API-Key for authentication.

5 Frontend prototype (hand sketches)

Figures 2–5 present the graphical user interface of the Gym Management System. Figure 2 shows the connection dialog where the user enters the API URL and X-API-Key. Figure 3 shows the main screen displaying members and statistics. Figure 4 presents the member creation form. Figure 5 shows the course management screen.

Figures 2–5 present the frontend prototype of the Gym Management System. Figure 2 shows the connection dialog where the user enters the API URL and X-API-Key. Figure 3 shows the main dashboard after a successful login. Figure 4 presents the form used to create or edit a gym member. Figure 5 shows the course management screen where courses and trainers can be managed.

6 Operation with Docker Compose

The application is deployed using Docker Compose.

The `docker-compose.yml` file starts two services:

- PostgreSQL database
- FastAPI backend

The database credentials and API configuration are stored in a `.env` file and are not included in the source code.

The backend is deployed on the lecture server using Docker Compose. After starting the containers, the FastAPI application is available and communicates directly with the PostgreSQL database.

7 Scope, milestones and risks

Scope.

The project contains 6 database tables (Member, Trainer, Course, Membership, Payment and CourseRegistration) and 8 REST API endpoints for managing members, courses and payments.

Milestones and schedule.

Week	Milestone	Est. hours
1	Design ER model and database schema	8
2	Implement PostgreSQL database and FastAPI backend	12
3	Develop frontend prototype and connect it to the API	10
4	Testing, documentation and final improvements	10

Risks.

Possible risks include delays during API development, database integration issues or unexpected bugs during frontend implementation. These risks will be reduced by testing each component separately before integration.

8 Acceptance criteria and testing

The following acceptance criteria define when the Gym Management System is considered complete and correctly implemented. Each criterion can be verified through functional testing.

Acceptance criteria.

- New members can be created, updated and deleted successfully.
- Courses are correctly linked to trainers and displayed through the API.
- Payments are stored correctly in the PostgreSQL database.
- Protected API endpoints reject requests without a valid X-API-Key.

Test plan.

- **Database:** Verify primary keys, foreign keys, NOT NULL constraints and referential integrity.
- **API:** Test all GET, POST, PUT and DELETE endpoints using FastAPI TestClient and Postman.
- **Business rules:** Verify member registration, course assignment and payment processing.

Effort budget: the whole term project (system, documentation and video) should fit into roughly **40 h of work** over at most **2 months**. Keep your scope within that budget.

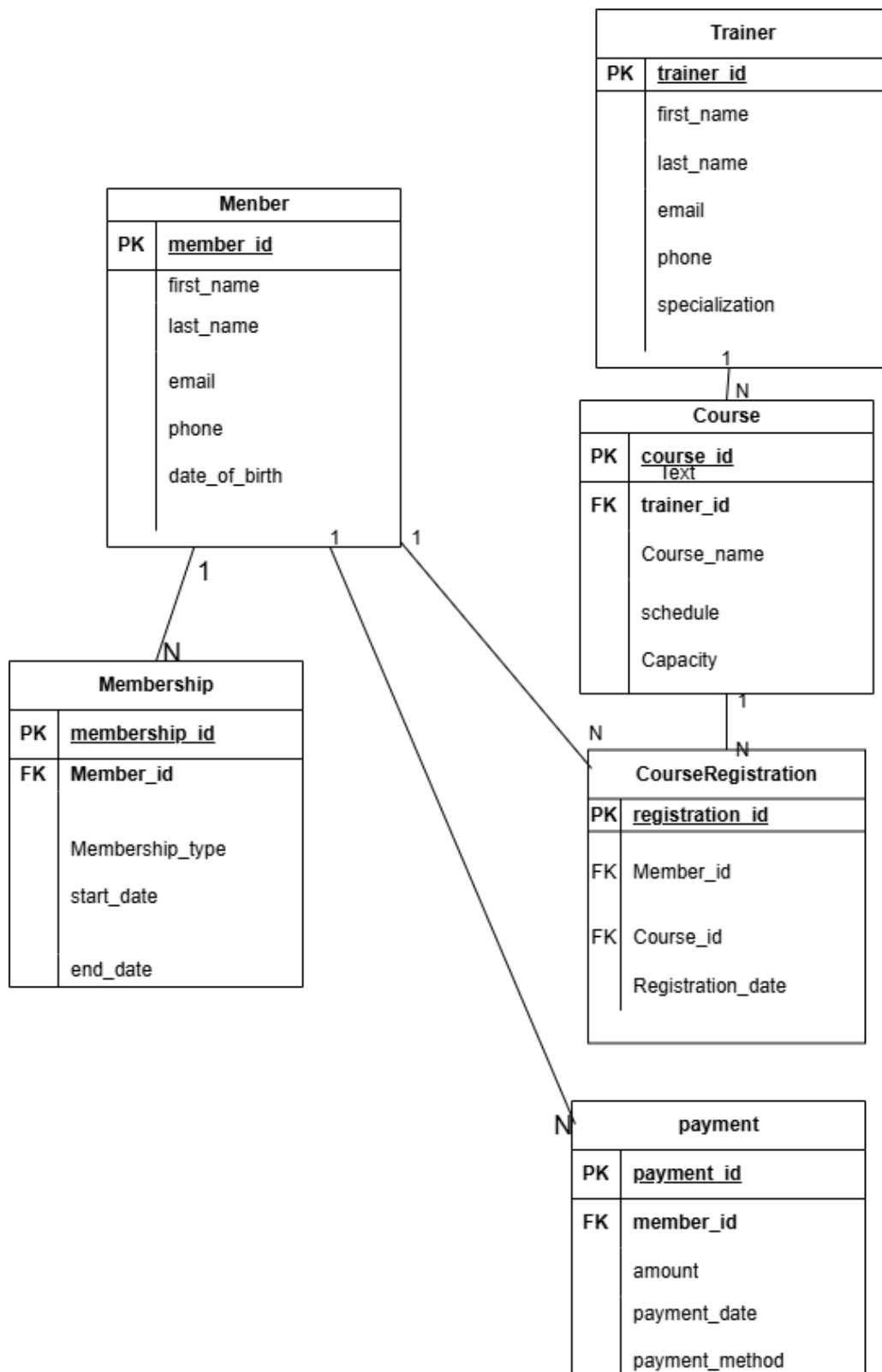


Abbildung 1: ER sketch of the «< domain »> data model: entity types, key attributes, relationships and cardinalities (at least one N:M).

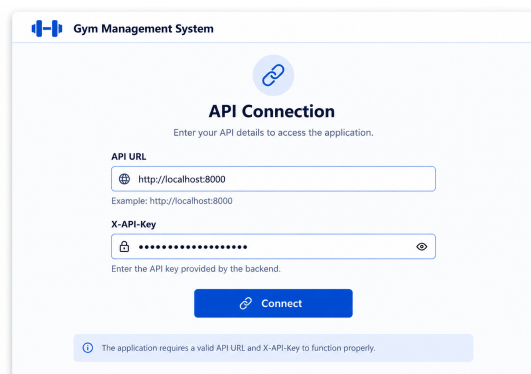


Abbildung 2: Startup connection dialog: API URL and X-API-Key authentication before accessing the application.

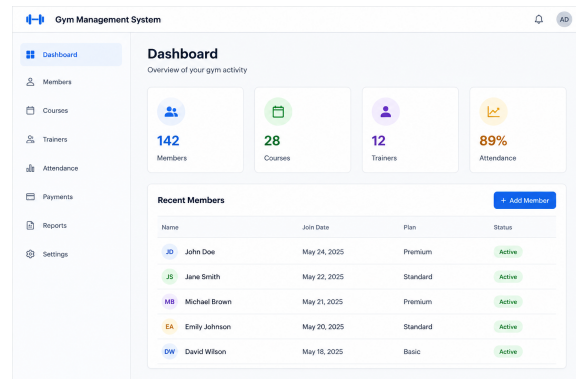


Abbildung 3: Main screen showing the dashboard with members, statistics and navigation menu.

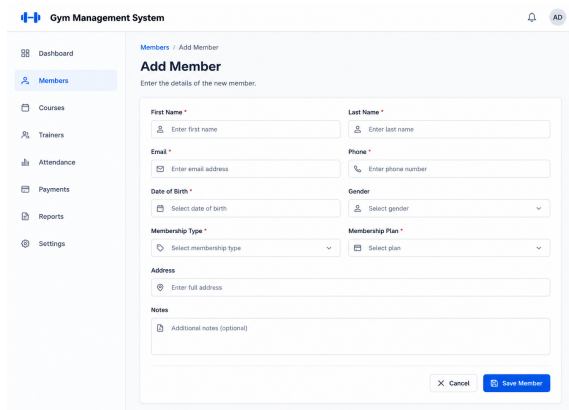


Abbildung 4: Detail form used to create or edit a gym member.

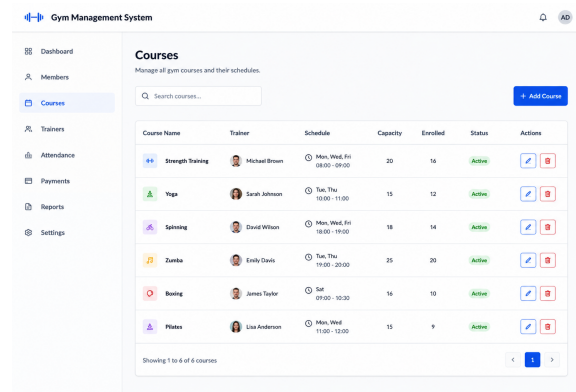


Abbildung 5: Course management screen displaying available courses and trainers.